

S⁴D

Nicole Ma Harsh Singh

We're Nicole and Harsh, and we're presenting S-four-D, a new state-of-the-art method for hierarchical speculative decoding.

We introduce the first reinforcement-learned routing policy for hierarchical speculative decoding, and set a new state of the art.

VIA-SD adds a slim intermediate verifier, but routes between tiers using confidence thresholds tuned by hand.

We present **three key contributions**:

1. A **learned per-token gating policy** that recasts tier-routing as a sequential decision problem, so a tiny q -free MLP replaces the fixed thresholds.
2. A **reward** that balances output quality against the latency of expensive verifier calls.
3. A **per-token RL formulation** with principled credit assignment (REINFORCE, then GRPO, then a per-token advantage).

The payoff: we match *lossless* accuracy (**0.91**) at **$4\times$ fewer** full-model calls, **$2.2\times$ faster** than greedy. We **Pareto-dominate** the paper's VIA-SD at roughly **$2\times$ its speedup** and **$2.3\times$ its accuracy**, and reach up to **$3.5\times$** when pushed for speed.

Here is the whole project on one slide. We built the first *learned* router for hierarchical speculative decoding. There are three contributions, and I'll give one slide to each. One: a learned per-token gate, a tiny network that decides, token by token, how much compute to spend. Two: a reward that balances output quality against cost. Three: a per-token reinforcement-learning setup that actually trains it well. And the payoff, which I'll keep coming back to: we match lossless accuracy, 0.91, at four times fewer expensive model calls. We Pareto-dominate VIA-SD, at roughly twice its speedup and over twice its accuracy. And pushed for speed, we go up to three and a half times faster. Let me unpack it.

The bottleneck: LLM decoding is slow and sequential

- Generating each token streams the entire model from memory, so LLM inference is memory-bandwidth bound and dominated by sequential forward passes.
- Speculative decoding fixes this: a cheap *drafter* guesses several tokens, and the big *verifier* checks them all in one parallel pass.
- Accepted guesses give many tokens per expensive forward. It is **lossless**, and runs about **1.4×** faster on our models.
- The one lever is the **acceptance rate**, because every rejected token forces a full-model step.

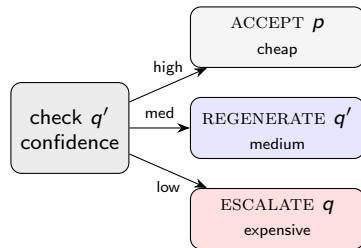
First, why speed matters at all. Generating a single token from a big model means reading every one of its weights out of memory. It's sequential, and it's the dominant cost of LLM inference, both latency and dollars. Speculative decoding is the standard fix. A small, cheap model guesses the next few tokens. The big model then checks all of those guesses in a single parallel pass. When the guesses are right, you got several tokens for the price of one big step, and the output is provably identical to normal decoding, so it's free quality-wise. On our models that's about 1.4 times faster. And the one thing that controls the whole method is the acceptance rate: every wrong guess wastes the speculation and forces a full, expensive step.

VIA-SD improves on vanilla SD

Vanilla SD makes a **binary** choice per token: accept the drafter, or reject and pay for the full model q .

VIA-SD adds a **slim middle verifier** q' (the target q with about 45% of its layers skipped), giving **three** choices. Borderline “middle-zone” tokens are handled cheaply by q' , so the full model is called far less often.

Since q' is the target verifying its *own* draft through a reduced copy of itself, this is the **Semi-Self** in S^4D .



Now, vanilla speculative decoding is all-or-nothing: you either accept the small model's guess, or you reject it and pay full price for the big model. VIA-SD's insight is that a lot of those rejected tokens are borderline. The draft was almost right, it just wasn't certain. So VIA-SD adds a middle option: the same big model, but with about 45 percent of its layers skipped. It's cheaper than the full model but still much smarter than the drafter. Now every token has three choices: accept the draft, regenerate it with this slim middle model, or escalate all the way to the full model. Borderline tokens get handled by the cheap middle tier, so you call the expensive full model far less often. And note: because q-prime is just the target verifying its own draft through a reduced copy of itself, this is the "Semi-Self" in our name. It's a genuinely good idea. But it raises one question: how does it decide which of the three tiers to use?

But it routes with thresholds that are picked by hand

VIA-SD makes the three-way choice with **two fixed confidence thresholds** on q' :

$$\boxed{\alpha_1 = 0.5} \quad \boxed{\alpha_2 = 0.3}.$$

- They are **global and static**: one cutoff for every token and every context.
- They are **brittle**, and must be re-tuned for each new model and task.
- They are **sub-optimal**: even after a full sweep, accuracy peaks at 0.50 on our setup.

Choosing a tier for each token, from its context, is a sequential decision problem. **So we learn it** — the gate teaches itself. That is the **Self-Taught** in S^4D .

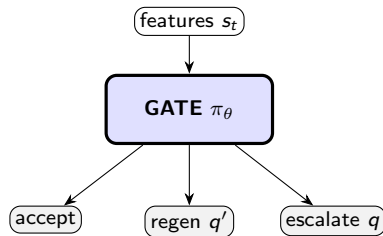
And here is the catch, the reason the whole project exists. VIA-SD makes that three-way decision with two numbers picked by hand: zero-point-five and zero-point-three. They're confidence thresholds. And they have three problems. They're global and static, the same cutoff for every token regardless of context. They're brittle, so you have to re-tune them for every new model and every new task. And they're simply sub-optimal: when we swept all nine threshold configurations, accuracy still capped at 0.50 on our setup. The mechanism, which I'll detail at the results, is that the slim verifier is miscalibrated, so a single global confidence cutoff cannot separate confident-and-correct tokens from confident-but-wrong ones, and the wrong ones it accepts compound into wrong answers. But step back. Deciding how much compute to spend on each token, based on that token's context, is a textbook sequential decision problem. So instead of hand-picking two numbers, the gate teaches itself the policy. That is the "Self-Taught" in S⁴D, and it gives us our three contributions.

Contribution 1/3: a learned per-token gating policy (MDP)

We recast tier-routing as a **Markov decision process**:

- **State** s_t : q -free features (drafter/ q' confidence, agreement, $\text{KL}(p\|q')$, position).
- **Action** $a_t \in \{\text{accept}, \text{regenerate}, \text{escalate}\}$.
- **Policy** $\pi_\theta(a_t | s_t)$: a tiny MLP.

The full model q is consulted only to *score* at train time. It is never a gate input, so the gate is free to run at inference.



Contribution one: the learned gate. We frame routing as a Markov decision process. The state, for each token, is a handful of cheap features: how confident the drafter and the slim model are, whether they agree, how far apart their distributions are, and where we are in the block. The action is which of the three tiers to use. And the policy is a tiny MLP. The single most important design choice is on the right of this slide: the features never include the full model. We only ever run the full model to compute rewards during training. So at inference time, the gate makes its decision instantly, from cheap signals, without running the expensive model just to decide. That's what makes it free to deploy: there's no second large model in the serving path.

Contribution 2/3: a reward balancing quality and cost

Two ingredients per token:

$$\text{quality: } \text{match}_t = \mathbf{1}[\hat{y}_t = \arg \max_v q(v \mid x_{<t})]$$

$$\text{cost: } \text{cost}_t = (t_{p_1} + t_{q'}/\gamma + \mathbf{1}[\text{ESCALATE}] t_q) / t_{q_1}$$

Reward (cost in units of one full-model step):

$$\text{trajectory: } R = \overline{\text{match}} - \lambda \overline{\text{cost}} \text{ (+ } r_c \text{ correct)} \qquad \text{per-token: } r_t = \text{match}_t - \lambda \text{cost}_t$$

λ is the quality-versus-speed knob. `match` is a *dense* proxy for the sparse true objective (final-answer correctness).

Contribution two: the reward, and this is where we pull from the writeup directly. There are two ingredients per token. Quality is simple: does the token we actually emit match the token the full model would have produced? That's the match term, and we compute it by running the full model at training time only. Cost is the per-action latency: the drafter step, plus the slim model amortized over the block, plus the full model, but only if we escalate, all normalized by the cost of one full-model step. The reward is just match minus lambda times cost. Lambda is the knob that trades accuracy for speed. Now the one honest subtlety, and we measured it: matching the full model token by token is only a proxy for what we truly care about, which is getting the final answer right. They are not the same thing. Ninety-eight and a half percent token match still left us at only 0.72 final accuracy, because a couple of slips can derail a whole chain of reasoning. That tension drives a lot of our analysis.

Contribution 3/3: per-token RL with credit assignment

Policy gradient (all variants, plus an entropy bonus $\eta \mathcal{H}[\pi_\theta]$):

$$\nabla_\theta J = \mathbb{E} \left[\sum_t A_t \nabla_\theta \log \pi_\theta(a_t | s_t) \right]$$

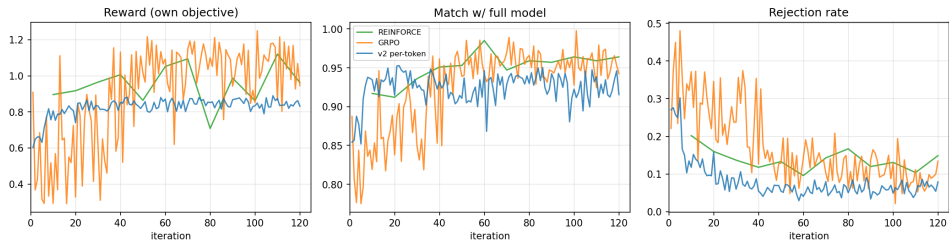
They differ only in the **advantage** A_t :

- **REINFORCE** (trajectory): $A = R - b$, with EMA baseline $b \leftarrow (1-\tau)b + \tau R$.
- **GRPO** (group): $A_i = \frac{R_i - \text{mean}_G R}{\text{std}_G R}$, with PPO-clip $\rho_t = \frac{\pi_\theta}{\pi_{\theta_{\text{old}}}}$ and KL to π_{ref} .
- **per-token (v2)**: $\hat{A}_t = \frac{r_t - \text{mean}_{\text{batch}} r}{\text{std}_{\text{batch}} r}$, local credit and the **lowest variance**.

Contribution three: how we actually optimize the gate. All three methods share the same skeleton, a policy gradient. In words: increase the probability of actions that did better than a baseline, and add an entropy bonus so it keeps exploring. The methods differ in exactly one thing: how they compute that “did better than baseline” term, the advantage. REINFORCE takes one reward for the whole sequence, subtracts a running average, and applies it to every decision. Simple, but very noisy. GRPO runs several rollouts of the same prompt and scores each against the group, which cancels out how hard the prompt is, and it’s the right tool for sparse rewards; it also adds PPO clipping and a KL leash to the starting policy for stability. Our per-token version gives every single decision its own advantage. Because routing is fundamentally a local choice, this is the lowest-variance signal and it trains best. The one-line lesson: credit assignment is the whole game here.

Training converges, and per-token (v2) is cleanest

RL training curves (0.5B→14B, GSM8K)



Quick proof that the training works. All three methods converge: reward climbs, agreement with the full model rises to about 0.95, and the rejection rate falls. The interesting part is the variance. GRPO, in orange, is jumpy, because it only gets one signal per group. REINFORCE, in green, is in the middle. And our per-token method, in blue, is the smoothest and drives rejections the lowest, exactly because every decision gets its own credit instead of sharing one reward across the whole sequence. One detail worth flagging: match plateaus around 0.95, not 1.0, because the q-free features can't perfectly flag every token the drafter will get wrong. That five-percent residual mismatch is exactly what compounds into the accuracy gap we discuss later. This is the evidence for the credit-assignment claim on the last slide.

Results: the learned gate dominates

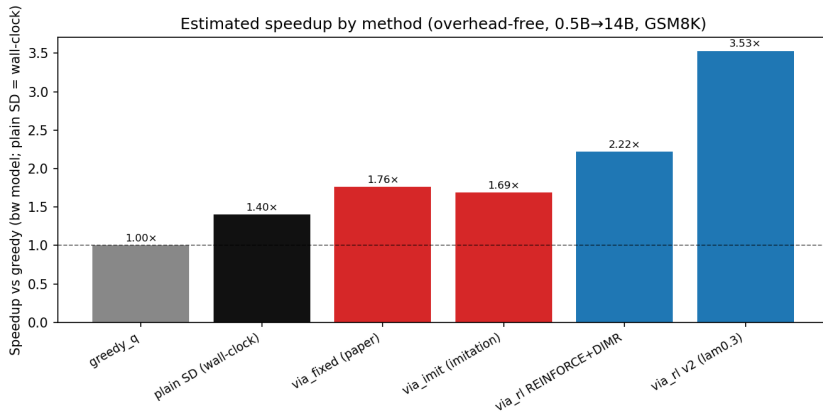
Method	GSM8K acc	q-calls/tok	speedup
greedy- q (reference)	0.887	1.000	1.00×
plain SD (standard, lossless)	0.907	0.259	1.40×
via_fixed (paper, tuned)	0.40	0.31	1.76×
via_imit (imitation only)	0.34	0.29	1.69×
via_rl REINFORCE	0.880	0.243	2.29×
via_rl GRPO	0.853	0.162	2.94×
via_rl REINFORCE+DIMR	0.907	0.250	2.22×
via_rl v2 per-token	0.713	0.104	3.53×

⇒ **S⁴D** is

state-of-the-art hierarchical speculative decoding.

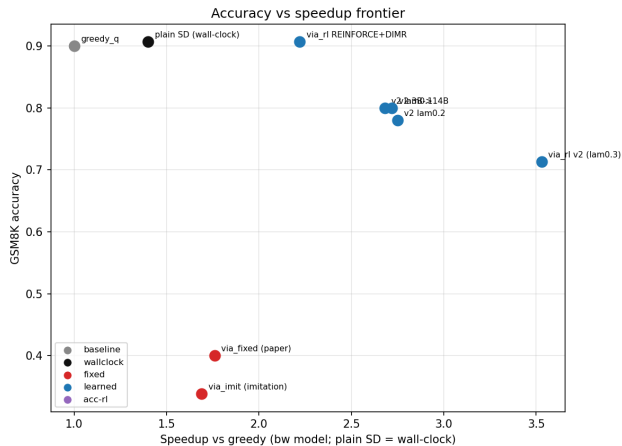
Here are the numbers, and I'll give the mechanism behind each row, not just the values. The top two are references: greedy decoding and lossless speculative decoding. They should be identical, because speculative decoding is exact; the tiny gap, 0.887 versus 0.907, is just bf16 numerical noise flipping a couple of borderline problems. Now the red rows, the hand-tuned thresholds, at 0.34 and 0.40. Here is why they're so low, and this is the key mechanistic point. The threshold accepts a token whenever a confidence score clears a fixed cutoff. But q-prime is a degraded model with about half its layers skipped, so its confidence is miscalibrated: it is frequently confident and wrong. A single global cutoff cannot separate confident-and-correct from confident-but-wrong, so it waves through a stream of locally-plausible but globally-wrong tokens. And on GSM8K the final answer is all-or-nothing, so even a two-or-three-percent per-token error rate compounds over a hundred-plus tokens into a wrong answer. That is also why a full nine-config sweep still caps at 0.50: there is no single cutoff that is right in every context. The imitation-only baseline is even worse, 0.34, because imitation learning adds exposure bias: it was trained on near-greedy states but drifts at its own states and over-accepts even more. The blue rows fix exactly this. RL trains on the policy's own rollouts, which kills the exposure bias, and the match reward teaches the gate to escalate precisely on the tokens where the drafter would err. REINFORCE-plus-DIMR recovers full accuracy, 0.907, matching lossless, because a better q-prime makes the middle tier trustworthy enough to reproduce the full model's output. GRPO drives the expensive-call count to the floor, 0.16 per token. And the per-token version

Latency: speedup by method



Same story, as latency. Greedy is the one-times baseline. Notice the hand-tuned gate, in red, is actually slow. It over-escalates to the full model, which defeats the whole purpose. Our learned gates, in blue, push the speedup up; the per-token version tops the chart by keeping escalation below the break-even point. The learned gate is the fastest of all the routing methods.

The accuracy and speed frontier



This one plot is the whole story: accuracy on the vertical axis, speed on the horizontal. Red, in the bottom-left, the hand-tuned methods, slow and inaccurate, completely dominated. Why are they dominated on *both* axes at once? One mechanism: a fixed cutoff is forced to do two jobs it can't do together. It over-escalates routine tokens, which makes it slow, and it over-accepts miscalibrated ones, which makes it inaccurate. A learned, context-aware gate escalates only where it matters, so it is both faster and more accurate. Blue, up and to the right, our learned gates. And the key point is that we don't just win at one setting. The lambda knob gives us an entire frontier. Dial it toward accuracy, around 0.80, or toward speed, ten times fewer expensive calls. Even better, look at the 3B-drafter point: a stronger drafter hits the same 0.80 accuracy with thirty-six times fewer full-model calls. That tells us the right lever is a better drafter, and this approach only gets stronger as drafters improve. Bottom line: we Pareto-dominate the hand-tuned baseline everywhere on this plot, which makes S⁴D the state of the art in hierarchical speculative decoding.

Impact: a new accuracy and efficiency frontier

- **Lossless quality, 4× cheaper.** We match greedy/lossless accuracy (**0.91**) at 4× **fewer** full-model calls, a 2.2× speedup, with *zero accuracy lost*.
- **A tunable frontier.** One knob gives 2.7× at -0.1 accuracy, or 3.5× at -0.19 , up to 10× **fewer** full-model calls.
- **Pareto-dominates VIA-SD.** About 2× its speedup *and* $+0.3$ to $+0.5$ accuracy. Where its fixed thresholds collapse to 0.40, our gate holds **0.71 to 0.91**.
- **Scales with the drafter.** A 3B drafter gives 36× **fewer** full-model calls at 0.80 accuracy, and it is a free-to-deploy tiny MLP.

S⁴D is the new state of the art in hierarchical speculative decoding.

Stop hand-picking thresholds. Let the model schedule its own compute.

So why does this matter, and I'm going to hit you with the numbers. First: we match lossless accuracy, 0.91, at four times fewer expensive model calls. That's a 2.2-times speedup with zero accuracy lost. Second: it's a tunable frontier, not a single point. 2.7 times faster for a tenth of a point of accuracy, or 3.5 times faster for about 0.19, up to ten times fewer expensive calls. Third, and this is the headline: we Pareto-dominate VIA-SD, at roughly twice its speedup and three-tenths to a half-point higher accuracy. Where its hand-tuned thresholds collapse to 0.40, we hold between 0.71 and 0.91. Fourth: it scales with the drafter. A 3B drafter gives thirty-six times fewer full-model calls, and it's free to deploy, a tiny network that never even touches the big model to make its decision. The takeaway is one sentence: stop hand-picking thresholds, and let the model schedule its own compute.

S⁴D

Self-Taught Semi-Self Speculative Decoding

The new state of the art in hierarchical speculative decoding.

Stop hand-picking thresholds. Let the model schedule its own compute.

Thanks! questions?

To wrap up: we took VIA-SD's brittle, hand-tuned thresholds and turned them into a learned policy. The result is a Pareto improvement: lossless quality at a fraction of the cost, dominating the baseline, free to deploy, and getting better as models improve. That is what makes S⁴D the state of the art in hierarchical speculative decoding. Stop hand-picking thresholds, and let the model schedule its own compute. Thank you, we're happy to take questions.

Ablation: which model should we scale?

Setup	GSM8K acc	q-calls/tok	note
0.5B → 14B (main)	0.88–0.91	0.10–0.25	baseline pair
3B → 14B (bigger <i>drafter</i>)	0.80	0.028	36× fewer full-model calls
0.5B → 32B (bigger <i>verifier</i>)	0.33	0.07	accuracy collapses

- A stronger **drafter** raises acceptance, so the gate escalates far less at similar accuracy.
- A bigger **verifier** with the same weak drafter widens the gap, and the accept-heavy gate emits tokens the target would reject.

The lever is the drafter, not the verifier.

Which model do you make bigger? We tested both, and the answer is counterintuitive. Make the *drafter* bigger, from 0.5B to 3B, and accuracy holds at 0.80 while expensive calls drop to 0.028 per token, thirty-six times fewer. The mechanism: a stronger drafter is right more often, so its tokens clear the verifier's bar, acceptance goes up, and the gate almost never needs to escalate. Now make the *verifier* bigger instead, from 14B to 32B, and accuracy collapses to 0.33. The mechanism there: the drafter is unchanged, so relative to a smarter 32B target its guesses are now wrong far more often, and our accept-heavy gate keeps emitting those drafter tokens even though the 32B would not. So a bigger target model actually hurts unless you also strengthen the drafter. The lever is the drafter.

Ablation: the slim verifier q' and the cost knob λ

- **Better q' via DIMR layer-skip search:** REINFORCE 0.88 $\rightarrow$ 0.91. A trustworthy regenerate tier lets the gate use q' instead of escalating, and the tokens come out right.
- **But q' quality is moot once the gate is accept-heavy** (the v2 policy routes around the regenerate tier entirely).
- **The λ knob is a real frontier:** $\lambda=0.1 \rightarrow 0.80$, $0.2 \rightarrow 0.78$, $0.3 \rightarrow 0.71$ accuracy.
- **Honest negative:** optimizing final-answer correctness *directly* (instead of the match proxy) degraded every warm-start we tried; the match-trained REINFORCE+DIMR (0.91) stays best.

Two more knobs we ablated. First, the quality of the slim middle model q-prime, which we build with the DIMR layer-skip search. A better q-prime lifts REINFORCE from 0.88 to 0.91. The reason: when the regenerate tier is trustworthy, the gate can send borderline tokens to q-prime instead of escalating, and they still come out correct. But notice the flip side: once a policy is accept-heavy, like our per-token v2, it barely touches the regenerate tier at all, so q-prime quality stops mattering. That's why DIMR helps the balanced policies but not the aggressive one. Second, the lambda knob is a genuine frontier, not a single point: 0.1, 0.2, 0.3 give 0.80, 0.78, 0.71 accuracy as we trade accuracy for speed. And the honest negative we promised: we tried optimizing the true final-answer correctness directly, instead of the token-match proxy, and in every configuration it degraded the policy we started from. The mechanism is that final-answer correctness is sparse, and the q-free features simply can't tell which specific token is about to derail the answer, so the gradient ends up just trading accuracy for cost. The match-trained REINFORCE-plus-DIMR, at 0.91, remains our best accuracy point.

Future work

- **Spec-spec decoding:** make the drafter itself speculative, a recursive hierarchy where the gate routes across *levels*.
- **KnapSpec:** treat verification as a **knapsack**, spending a fixed budget of expensive calls on the highest-value tokens (value is correctness impact, weight is cost).
- **Correctness-floor RL:** minimize cost *subject to* an accuracy floor, which avoids the reward-hacking collapse.
- **Scale:** bigger drafters, more tasks, and real-engine wall-clock (static cache, CUDA graphs).

Three directions we're excited about. First, spec-spec decoding: the drafter itself is slow, so make it speculative too, a recursive hierarchy of drafters, and the gate now routes across levels, not just three tiers. Second, what we call KnapSpec: you have a fixed budget of expensive calls per sequence, so spend it like a knapsack, putting your verification budget on the tokens that most change the final answer. That directly attacks the proxy gap I showed earlier. Third, a correctness-floor objective: minimize cost subject to staying above an accuracy floor, which avoids the failure mode where naively optimizing for correctness collapses the policy. And then scale it up: bigger drafters, more tasks, and real wall-clock on a production inference engine.